

PHISHING WEBSITES IDENTIFICATION USING phiKitA

A Mini Project report submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE AND ENGINEERING
by

B. AMANI	(22S41A0519)
K. ARCHANA	(22S41A0556)
B. RAKESH	(22S41A0517)
D.BHAVANA	(22S41A0534)

Under the Guidance of
Dr. MD. SIRAJUDDIN
Associate Professor



Department of Computer Science & Engineering
VAAGESWARI COLLEGE OF ENGINEERING

Accredited by NAAC with A+ Grade
(Affiliated to JNTU Hyderabad & Approved by AICTE, New Delhi)
Ramakrishna colony, Karimnagar-505527.

May-2025

Department of Computer Science & Engineering
VAAGESWARI COLLEGE OF ENGINEERING

Accredited by NAAC with A+ Grade
(Affiliated to JNTU Hyderabad & Approved by AICTE, New Delhi)
Ramakrishna colony, Karimnagar-505527.



CERTIFICATE

This is to certify that the project report entitled “**PHISHING WEBSITES IDENTIFICATION USING PhiKitA**” submitted by the following students in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in CSE and is a Bonafide record of the work performed by.

B. AMANI	(22S41A0519)
K. ARCHANA	(22S41A0556)
B. RAKESH	(22S41A0517)
D.BHAVANA	(22S41A0534)

Internal Guide

Dr. MD. SIRAJUDDIN
Associate Professor

Head of the department

Dr. N. CHANDRAMUOLI
Professor

Principal
Dr. CH. SRINIVAS

External Examiner

DECLARATION

We hereby declare that the project titled “**PHISHING WEBSITES IDENTIFICATION USING phiKitA**” submitted to Vaageswari College of Engineering, affiliated to Jawaharlal Nehru Technological University Hyderabad (JNTUH) for the award of the Degree of Bachelor of technology in CSE is a result of original research carried-out in this work. It is further declared that the report or any part thereof has not been previously submitted to any University or Institute for the award of degree or diploma.

1. Name of the Student: B. AMANI

Hall Ticket Number:22S41A0519

2. Name of the Student: K. ARCHANA

Hall Ticket Number:22S41A0556

3. Name of the Student: B. RAKESH

Hall Ticket Number: 22S41A0517

4. Name of the Student: D. BHAVANA

Hall Ticket Number: 22S41A0534

Date:

ACKNOWLEDGEMENT

We wish to pay our sincere thanks to **Dr. CH. SRINIVAS**, Principal, Vaageswari College of Engineering, Karimnagar, for providing all required facilities and his support during the Project Work.

We would like to thank **Dr. N. CHANDRAMOULI**, Professor and HOD of the Computer Science and Engineering Department for his valuable suggestions during the Project Work.

We sincerely extend our thanks to the mini project guide **Dr. MD. SIRAJUDDIN** Associate Professor of Computer Science and Engineering Department, for sparing his valuable in guiding the Minor project work and giving feedback with many useful suggestions during the minor project work.

We are also conveying our heartfelt thanks to the Institute authority, Department, Library, and Laboratory staff of Vaageswari College of Engineering for their co-operation during our Project Work. We thank our beloved friends for their help and encouragement regarding the concepts and presentation.

B. AMANI	(22S41A0519)
K. ARCHANA	(22S41A0556)
B. RAKESH	(22S41A0517)
D.BHAVANA	(22S41A0534)

ABSTRACT

Recent studies have shown that phishers are using phishing kits to deploy phishing attacks faster, easier and more massive. Detecting phishing kits in deployed websites might help to detect phishing campaigns earlier. To the best of our knowledge, there are no datasets providing a set of phishing kits that are used in websites that were attacked by phishing. In this work, we propose PhiKitA, a novel dataset that contains phishing kits and also phishing websites generated using these kits. We have applied MD5 hashes, fingerprints, and graph representation DOM algorithms to obtain baseline results in PhiKitA in three experiments: familiarity analysis of phishing kit samples, phishing website detection and identifying the source of a phishing website. In the familiarity analysis, we find evidence of different types of phishing kits and a small phishing campaign. In the binary classification problem for phishing detection, the graph representation algorithm achieved an accuracy of 92.50%, showing that the phishing kit data contain useful information to classify phishing. Finally, the MD5 hash representation obtained a 39.54% F1 score, which means that this algorithm does not extract enough information to distinguish phishing websites and their phishing kit sources properly

LIST OF FIGURES

S.NO	TABLES/FIGURES	PAGE NO'S
1	System Architecture	12
2	UML Diagrams	13-19
	2.1 Use Case Diagram	14
	2.2 Class Diagram	15
	2.3 Sequence Diagram	16
	2.4 Collaboration Diagram	17
	2.5 Activity Diagram	17
	2.6 Component Diagram	18
	2.7 Deployment Diagram	19
	2.8 ER Diagram	19
	2.9 Data Dictionary/Dataset	20-21
3	About Python	22-41
4	Screenshots	42-45

INDEX

TOPICS	Page No's
CHAPTER-1: INTRODUCTION	1
CHAPTER-2: LITERATURE SURVEY	3
CHAPTER-3: REQUIREMENT ANALYSIS	6
3.1 Existing System	
3.2 Proposed System	
3.3 System Requirements	
3.4 Feasibility Study	
CHAPTER-4: SYSTEM DESIGN	12
4.1 System Architecture	
4.2 UML Diagrams	
CHAPTER-5: IMPLEMENTATION	22
5.1 Module	
5.2 Technology	
5.3 System Testing	
CHAPTER-6: RESULTS/DISCUSSIONS	42
CHAPTER-7: CONCLUSION	46
REFERENCES	

CHAPTER-1

INTRODUCTION

Phishing remains one of the most prevalent and damaging forms of cyberattacks, with threat actors continuously evolving their techniques to bypass traditional security mechanisms. One of the emerging trends in this domain is the use of phishing kits—pre-packaged, easy-to-deploy toolsets that enable even low-skilled attackers to launch sophisticated phishing campaigns with minimal effort. These kits streamline the creation of fake websites and allow attackers to replicate legitimate interfaces efficiently, making detection more challenging. Despite their growing use, there is a notable lack of publicly available datasets that focus on phishing kits and their direct correlation with phishing websites. Such datasets could be instrumental in developing early detection systems and understanding the structure and propagation of phishing campaigns. Addressing this gap, our research introduces PhiKitA, a novel dataset comprising both phishing kits and the phishing websites they generate. This work explores how information derived from phishing kits can enhance detection methods and uncover campaign patterns through various analytical approaches.

Phishing attacks have long been a persistent and evolving threat in the cybersecurity landscape. These attacks trick unsuspecting users into divulging sensitive information such as usernames, passwords, and credit card numbers by masquerading as trustworthy entities in electronic communications. Despite continuous efforts to educate users and strengthen technical defenses, phishing remains one of the most successful forms of cybercrime, largely due to the evolving sophistication of attack vectors and the ease with which such attacks can be executed. A particularly troubling development in recent years is the widespread use of phishing kits—pre-packaged sets of tools and files that simplify the creation and deployment of phishing websites. Phishing kits enable attackers to replicate the look and feel of legitimate websites without requiring in-depth programming skills. Typically distributed on underground forums or hacking communities, these kits often come with HTML files, CSS, images, JavaScript, and PHP scripts that process the stolen information. More advanced kits also include evasion mechanisms to bypass security filters or detection systems. The commoditization of phishing through these kits has led to an increase in both the volume and diversity of phishing attacks, enabling even novice cybercriminals to orchestrate large-scale campaigns with minimal effort.

While significant research has been conducted to detect phishing websites using URL-based features, visual similarity, and machine learning algorithms, relatively little attention has been paid to the detection and analysis of phishing kits themselves. Phishing kits serve as the foundation of many phishing websites, and studying them can provide valuable insights into the structure, origin, and spread of phishing campaigns. Moreover, detecting the presence of a phishing kit on a

compromised website could act as an early warning signal, enabling faster mitigation and takedown efforts.

Despite their importance, there is currently a lack of publicly available datasets that capture phishing kits and their associated websites. Most phishing-related datasets focus on blacklisted URLs or screenshots of phishing pages, which limits the ability of researchers to understand the underlying tools and methodologies used by attackers. To address this gap, we propose PhiKitA (Phishing Kit Analysis), a novel dataset that includes not only phishing kits but also the phishing websites generated using those kits. This dataset represents a significant step toward deeper, data-driven research into phishing operations.

The PhiKitA dataset was compiled by collecting phishing kits from various sources and linking them to the corresponding deployed phishing websites. This linkage allows for comprehensive analysis across multiple dimensions: identifying reused code and templates, detecting patterns in phishing infrastructure, and even tracing the origin of campaigns. In our study, we applied multiple algorithms to extract baseline results from the PhiKitA dataset. These include MD5 hashing for kit identification, fingerprinting for structural similarity, and graph-based representations of the Document Object Model (DOM) for phishing website classification.

In the first experiment—familiarity analysis—we examined how often phishing kits are reused and whether certain kits are part of coordinated campaigns. This analysis revealed a number of distinct phishing kit families and provided evidence of a small-scale phishing campaign, suggesting that kit reuse is common and can be a valuable clue in identifying related attacks.

In the second experiment, we framed phishing detection as a binary classification problem using features derived from the phishing kits. Notably, the graph representation of DOM structures yielded an accuracy of 92.50%, demonstrating that structural data embedded in phishing kits is highly informative for detecting phishing websites. This approach offers a promising direction for the development of robust phishing detection tools that go beyond superficial visual or URL-based indicators.

Lastly, we explored the task of identifying the specific phishing kit used to generate a given phishing website. Here, the MD5 hash-based approach achieved only a 39.54% F1 score, highlighting its limitations in capturing nuanced differences between kits. This result underscores the need for more sophisticated techniques that can effectively model the semantic and functional characteristics of phishing kits.

Overall, the PhiKitA dataset opens up new avenues for cybersecurity research by providing a rich resource for studying phishing kits and their real-world impact. By shifting the focus from the front-facing phishing websites to the tools behind them, this work aims to enhance our understanding of phishing campaigns and support the development of proactive defense mechanisms.

CHAPTER-2

LITERATURE SURVEY

TITLE: PhiKitA: Phishing Kit Attacks Dataset for Phishing Websites Identification

AUTHORS: Felipe Castaño, Eduardo Fidalgo Fernandez, Rocío Alaiz-Rodríguez, Enrique Alegre

ABSTRACT: Recent studies indicate that phishers increasingly use phishing kits to facilitate faster, easier, and more extensive phishing attacks. Detecting these kits on deployed websites could enable earlier identification of phishing campaigns. Currently, no datasets include both phishing kits and the websites they generate. To address this, we introduce PhiKitA, a novel dataset containing phishing kits and corresponding phishing websites. We utilized MD5 hashes, fingerprints, and graph representation DOM algorithms to perform three experiments: analyzing the familiarity of phishing kit samples, detecting phishing websites, and identifying the source of these websites. Our familiarity analysis revealed various types of phishing kits and a small-scale phishing campaign. For binary classification in phishing detection, the graph representation algorithm achieved 92.50% accuracy, demonstrating the usefulness of phishing kit data in identifying phishing. However, the MD5 hash representation yielded only a 39.54% F1 score, indicating insufficient information extraction to distinguish phishing websites and their sources accurately.

TITLE: PhiKitA: Phishing Kit Attacks Dataset for Phishing Website Detection

AUTHORS: Felipe Castaño, Eduardo Fidalgo Fernandez, Rocío Alaiz-Rodríguez, Enrique Alegre

ABSTRACT: Phishing kits simplify the deployment of phishing attacks, making them more widespread and efficient. Early detection of these kits can significantly mitigate the impact of phishing campaigns. Despite the critical need, there is a lack of comprehensive datasets that include both phishing kits and the phishing websites they generate. PhiKitA addresses this gap by providing such a dataset. We conducted experiments using MD5 hashes, fingerprints, and graph representation DOM

algorithms. Our analysis covered three aspects: the familiarity of phishing kit samples, the detection of phishing websites, and the identification of their sources. The familiarity analysis highlighted different phishing kits and a minor phishing campaign. The graph representation algorithm achieved a 92.50% accuracy in binary classification for phishing detection, indicating that phishing kit data are valuable for classification. In contrast, the MD5 hash approach achieved a 39.54% F1 score, suggesting it does not effectively differentiate between phishing websites and their sources.

TITLE: Phishing Kits: Dataset Development and Detection Algorithms

AUTHORS: Felipe Castaño, Eduardo Fidalgo Fernandez, Rocío Alaiz-Rodríguez, Enrique Alegre

ABSTRACT: Phishing kits enable phishers to launch attacks quickly and efficiently. Detecting these kits early can provide significant advantages in combating phishing. We introduce PhiKitA, the first dataset to include phishing kits and the phishing websites created with them. Using MD5 hashes, fingerprints, and graph representation DOM algorithms, we established baseline results through three experiments: analyzing phishing kit sample familiarity, detecting phishing websites, and identifying the origin of these websites. Our familiarity analysis uncovered various types of kits and a minor phishing campaign. The graph representation algorithm achieved a notable 92.50% accuracy in phishing detection, underscoring the importance of phishing kit data. However, the MD5 hash approach, with a 39.54% F1 score, failed to provide sufficient differentiation.

TITLE: Enhancing Phishing Detection through Phishing Kit Data: The PhiKitA Dataset

AUTHORS: Felipe Castaño, Eduardo Fidalgo Fernandez, Rocío Alaiz-Rodríguez, Enrique Alegre

ABSTRACT: Phishers increasingly use phishing kits to streamline and amplify their attacks. Early detection of these kits is vital for thwarting phishing campaigns. The absence of datasets encompassing phishing kits and related phishing websites prompted the creation of PhiKitA. This dataset includes phishing kits and the websites they produce. We evaluated PhiKitA using MD5 hashes, fingerprints, and graph representation DOM algorithms in three experiments: phishing kit sample familiarity analysis, phishing website detection, and source identification. The analysis revealed different phishing kits and a small campaign. The graph representation algorithm's 92.50% accuracy in phishing detection confirms the

relevance of phishing kit data, while the MD5 hash method's 39.54% F1 score indicates its limitations

TITLE: elipe Castaño, Eduardo Fidalgo Fernandez, Rocío Alaiz-Rodríguez, Enrique Alegre

AUTHORS: Zangróniz R, Martínez-Rodrigo A, Pastor JM

ABSTRACT: The deployment of phishing attacks is increasingly facilitated by phishing kits, which allow for quicker and larger-scale operations. Early detection of these kits can significantly enhance the response to phishing campaigns. PhiKitA is a pioneering dataset that includes both phishing kits and the phishing websites generated by them. Our study applied MD5 hashes, fingerprints, and graph representation DOM algorithms across three experiments: phishing kit sample familiarity analysis, phishing website detection, and source identification. The familiarity analysis identified various phishing kits and a minor campaign. The graph representation algorithm achieved an accuracy of 92.50% in detecting phishing, highlighting the value of phishing kit data. Conversely, the MD5 hash method, with a 39.54% F1 score, was less effective in distinguishing between phishing websites and their sources.

CHAPTER-3

REQUIREMENT ANALYSIS

3.1 EXISTING SYSTEM:

Cova analyzed phishing kits by tracking the destination of the stolen information. First, they gathered the phishing kits from distribution sites or downloaded them by checking the directory contents of already reported phishing websites. The authors collected around 500 phishing kits, and after the analysis process, they discovered that many samples contained backdoors that send the stolen data back to the phisher and the original author.

Oest et al. studied the time response of anti-phishing groups' blocklists against evasion techniques using filters found in real phishing kits. The authors measured how cloaking techniques on phishing kits affect the timeliness of blocklisting phishing websites using sterilized phishing that contains different cloaking methods. The phishing websites were reported to anti-phishing groups and wait blocklisting time response. The dataset used in this experiment contained 2.380 spoofed PayPal login pages, and the authors reported that only 23% of cloaked websites were blocklisted against 49, 9% of websites without cloaking

In a later work, Oest et al. found that phishing kits are a key component of phishing attacks when they studied their life cycle. The authors monitored web events over the internet, processing the ones related to phishing websites. Finally, the authors reported that a phishing campaign takes 21 hours, and at least 7, 42% of the victims provide their Cova analyzed phishing kits by tracking the destination of the stolen information. First, they gathered the phishing kits from distribution sites or downloaded them by checking the directory contents of already reported phishing websites. The authors collected around 500 phishing kits, and after the analysis process, they discovered that many samples contained backdoors that send the stolen data back to the phisher and the original author.

Oest et al. studied the time response of anti-phishing groups' blocklists against evasion techniques using filters found in real phishing kits. The authors measured how cloaking techniques on phishing kits affect the timeliness of blocklisting phishing websites using sterilized phishing that contains different cloaking methods. The phishing websites were reported to anti-phishing groups and wait blocklisting time response. The dataset used in this experiment contained 2.380 spoofed PayPal login pages, and the

authors reported that only 23% of cloaked websites were blocklisted against 49, 9% of websites without cloaking.

In a later work, Oest et al. found that phishing kits are a key component of phishing attacks when they studied their life cycle. The authors monitored web events over the internet, processing the ones related to phishing websites. Finally, the authors reported that a phishing campaign takes 21 hours, and at least 7, 42% of the victims provide their credentials in that time window. The results presented in this work were extracted from a dataset with 19.359.676 events related to 404.628 different phishing URLs.

Britt et al. proposed one of the first methods that use phishing kits as a resource of information to identify phishing attacks. The authors used MD5 values to represent the similarity between the two samples by counting the overlapped files inside them. Later, they created groups of phishing website samples by comparing the samples' similarity to a specific phishing kit. The clustering algorithm found 22.904 clusters, where 14.129 of those clusters contain phishing websites assigned to a brand, showing a highly consistent brand grouping. The University of Alabama at Birmingham (UAB) phishing Data Mine group collected the dataset used, which contains 265.611 potential phishing websites. Although this work does not use information about phishing kits, it is based on the idea that these phishing websites were deployed using phishing kits and therefore have similar patterns and characteristics.

To detect phishing website attacks, Orunsolu and Sodiya presented an approach that uses phishing kit features. The method comprises a Signature Detection Module (SDM) that relies on 18 extracted features. These features are divided into HTML source, URL source, and phishing kit-related information. The phishing kit features include information such as hexadecimal obfuscation, toolkit names or URLs. Once the feature vector is extracted, the authors used it as an input to a Naive Bayes classifier reporting 98% accuracy on a dataset of 258 kits generated by websites. To perform these experiments, Orunsolu and Sodiya manually built the dataset, which involved two steps. First, ethical hackers and computer security students used five phishing kits to create 258 phishing websites that did not represent the conditions of a real attack. Then, in the second step, the authors collected 200 samples of phishing and legitimate websites on the internet between September and December 2014.

As a phishing identification technique, Tanaka et al. used a website structure signature of phishing kits. This signature is created by analyzing the Web Access Log generated when users access a landing page. A sample is classified as a phishing website if it reaches a structural similarity score of 0.5 or higher using the Jaccard coefficient compared to the previously collected phishing kit structural scores. The dataset was built

following two steps: First, the authors generated 49 phishing websites using phishing kits for the comparison base. Second, the authors collected 18.798 samples from July 2019 to March 2020 on PhishTank. They did not report any matching results, such as accuracy or F1-Score, since there is no way to relate the samples used for the comparison base with the samples collected in the second step. Instead, authors reported 1.742 phishing sites with similar structures to the comparison base, and after a manual revision, they determined that 95% of those samples were indeed similar.

Feng et al. [28] used web structure analysis from HTML sources to identify phishing websites. The authors addressed this problem using a clustering technique since phishers use phishing kits to deploy many phishing attacks. For this reason, the attacks from the same phishing kit may contain similar web structures. The method consists of three steps. First, the extraction of a feature vector with the HTML Document Object Model (DOM) information. Second, the authors grouped the samples by similarity and generated a feature vector from all the samples belonging to a single group. Finally, the feature vector for each group is compared against the fingerprint of websites to obtain a binary classification. To evaluate their method, they collected a dataset of 10.992 legitimate websites and 10.994 phishing websites. They concluded that this method could identify phishing website familiarity detect phishing attacks more efficiently than other methods. However, they did not report any comparison results, such as accuracy or F1-Score, since their dataset does not contain a ground truth between phishing kits and phishing websites..

Disadvantages:

Phishing detection methods are complex to test due to the difficulty of obtaining representative datasets. This is related to the changing nature of phishing attacks and the sensitivity of the data itself. Authors usually collect the data by themselves, considering the requirements of their proposed method. Then, they present the performance of the algorithm but do not release the collected data. All these reasons make comparing the performance of the literature methods a complex task, as they could be tested under certain conditions introduced by the decisions made in the creation process of the dataset.

4.2 PROPOSED SYSTEM:

We propose a methodology for collecting datasets that guarantees that the provided phishing websites are related to their phishing kit source. Using this methodology, we avoid the particular conditions introduced to the data by the decisions made by authors when creating phishing websites. We also guarantee the relationship between phishing kits and phishing website attacks as they are collected in the same process.

- We present PhiKitA, the first dataset, up to our knowledge, with a ground truth that is correct, presenting an accurate association between phishing kits and real phishing websites on the Internet. PhiKitA contains 510 phishing kit samples, 859 phishing website attacks and 1141 legitimate samples, and traces of a phishing campaign.
- We evaluate three different algorithms from the literature comparing their results on PhiKitA. For the first time, we evaluate the performance of these algorithms in three different experimental setups: familiarity analysis, phishing detection and multi-class classification to detect the source of a phishing website.

Advantages:

- The proposed system overcomes the previous drawbacks by presenting a methodology for collecting datasets where the phishing websites are clearly associated with their phishing kit source. Using this methodology, we created and made publicly available PhiKitA, a dataset containing phishing kits, phishing websites created by them and even traces of a phishing campaign.
- We also evaluated and compared the performance of several classification and clustering algorithms from the literature in our presented dataset.

3.3 SYSTEM REQUIREMENTS

3.3.1 FUNCTIONAL REQUIREMENTS

Functional requirements will vary for different types of software. For example, functional requirements for a website or mobile application should define user flows and various interaction scenarios. The major modules of the project are

1. Analytics Operator
2. Remote Client

3.3.2 NON-FUNCTIONAL REQUIREMENTS

Nonfunctional requirements are not related to the system's functionality but rather define how the system should perform. They are crucial for ensuring the system's usability, reliability, and efficiency, often influencing the overall user experience. We'll describe the main categories of nonfunctional requirements in detail further on

HARDWARE REQUIREMENTS:

- System : i3
- Hard Disk : 40 GB.
- Floppy Drive : 1.44 Mb.
- Monitor : 15 VGA Colour.
- Mouse : Logitech.
- Ram : 512 Mb.

SOFTWARE REQUIREMENTS:

- Operating system : Windows 7 Ultimate.
- Coding Language : Python. • Front-End Python.
- Back-End : Django-ORM
- Designing : Html, css, javascript.
- Data Base : MySQL (WAMP Server).

3.4 FEASIBILITY STUDY

The feasibility of the project is analyzed in this phase and business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the company.

Three key considerations involved in the feasibility analysis are

- ECONOMICAL FEASIBILITY
- TECHNICAL FEASIBILITY
- SOCIAL FEASIBILITY

ECONOMICAL FEASIBILITY

This study is carried out to check the economic impact that the system will have on the organization. The amount of fund that the company can pour into the research and development of the system is limited. The expenditures must be justified. Thus the developed system as well within the budget and this was achieved because most of the technologies used are freely available.

TECHNICAL FEASIBILITY

This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. This will lead to high demands on the available technical resources. This will lead to high demands being placed on the client.

SOCIAL FEASIBILITY

The aspect of study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The user must not feel threatened by the system, instead must accept it as a necessity. His level of confidence must be raised so that he is also able to make some constructive criticism, which is welcomed, as he is the final user of the system.

CHAPTER-4

SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE:

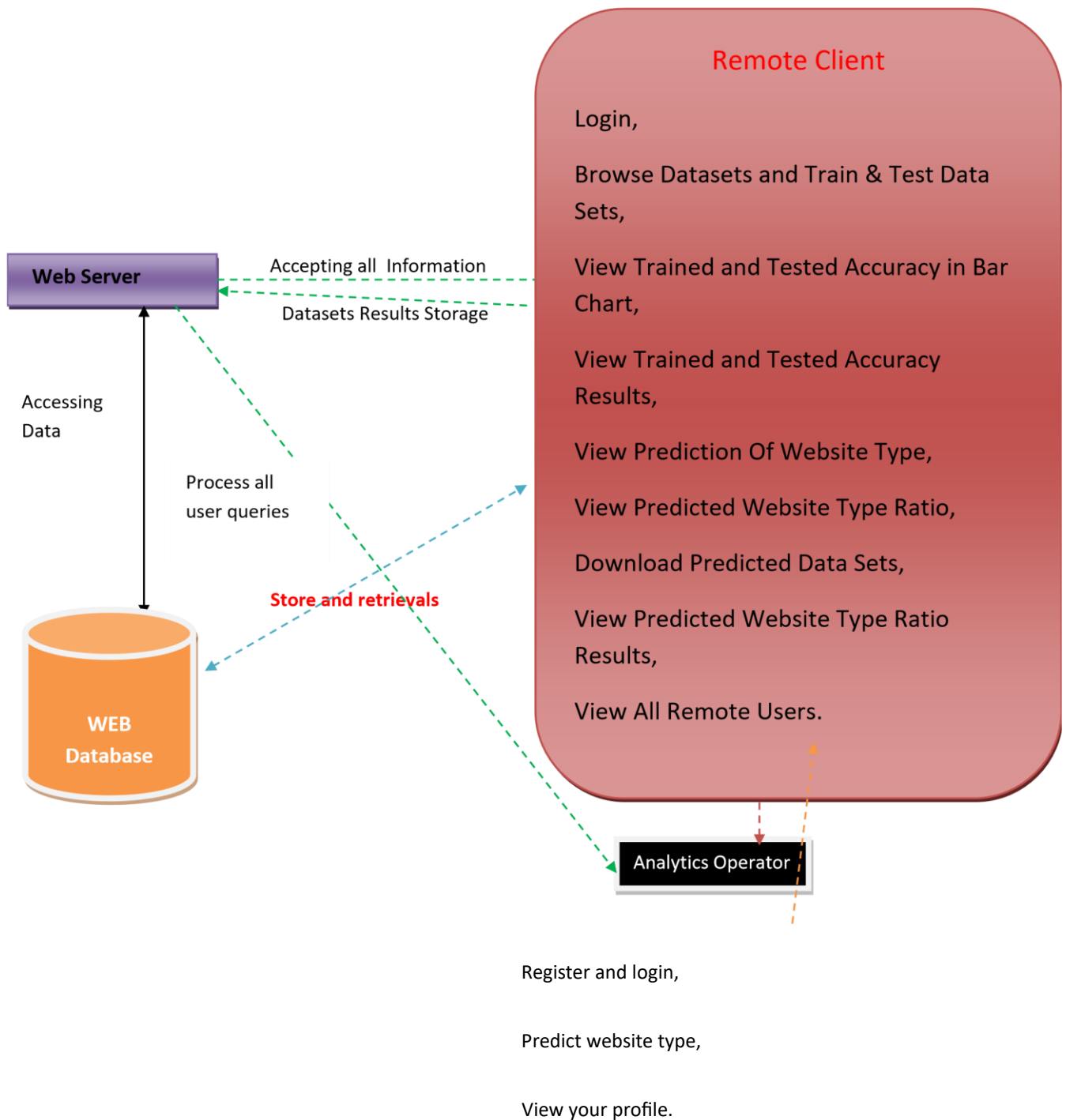


Fig 4.1: System Architecture

4.2 UML DIAGRAMS

UML stands for Unified Modeling Language. UML is a standardized general-purpose modeling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group.

The goal is for UML to become a common language for creating models of object oriented computer software. In its current form UML is comprised of two major components: a Meta-model and a notation. In the future, some form of method or process may also be added to; or associated with, UML.

The Unified Modeling Language is a standard language for specifying, Visualization, Constructing and documenting the artifacts of software system, as well as for business modeling and other non-software systems.

The UML represents a collection of best engineering practices that have proven successful in the modeling of large and complex systems.

The UML is a very important part of developing objects oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects.

GOALS:

The Primary goals in the design of the UML are as follows:

1. Provide users a ready-to-use, expressive visual modeling Language so that they can develop and exchange meaningful models.
2. Provide extendibility and specialization mechanisms to extend the core concepts.
3. Be independent of particular programming languages and development process.
4. Provide a formal basis for understanding the modeling language.
5. Encourage the growth of OO tools market.
6. Support higher level development concepts such as collaborations, frameworks, patterns and components.
7. Integrate best practices.

4.2.1 USE CASE DIAGRAM:

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.

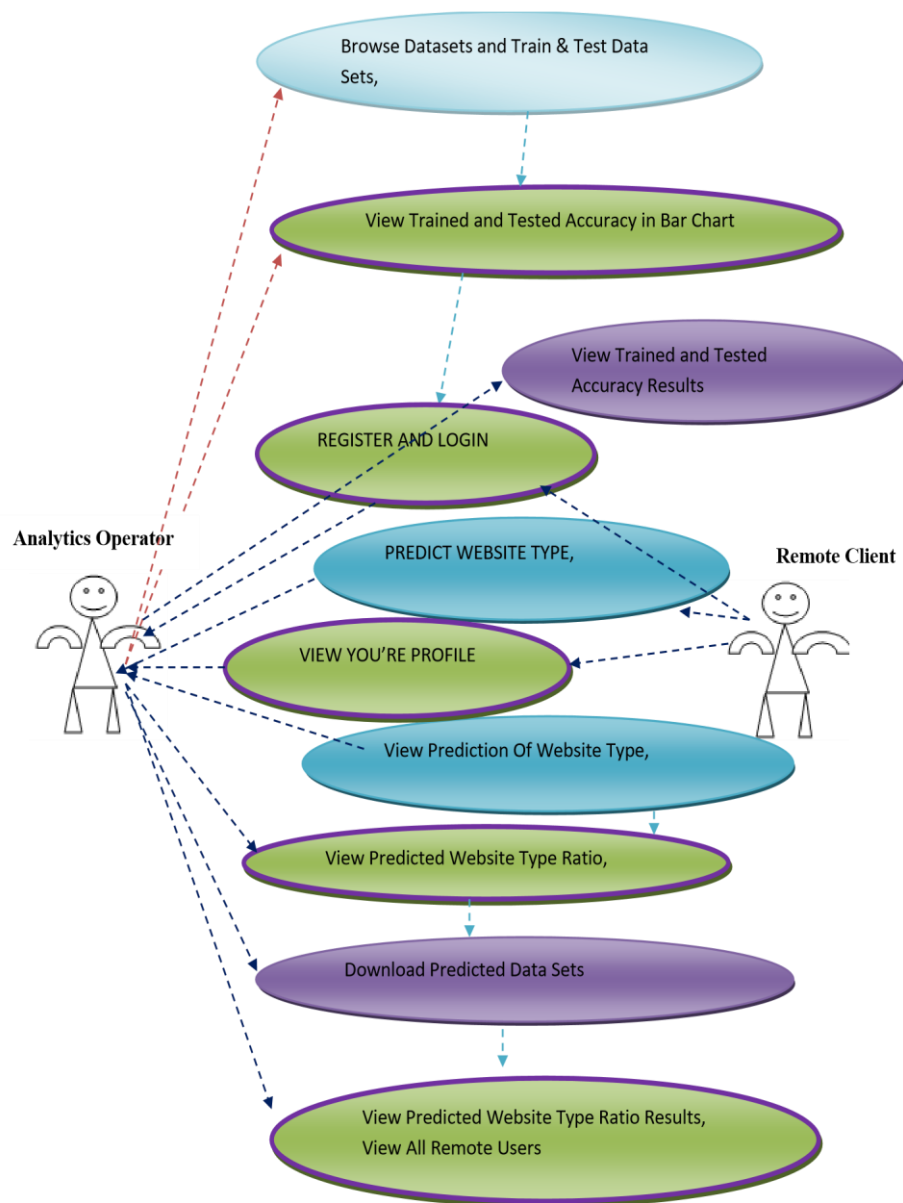


Fig 4.2.1: UseCase Diagram

4.2.2 CLASS DIAGRAM:

In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which Class contains information.

Analytics Operator

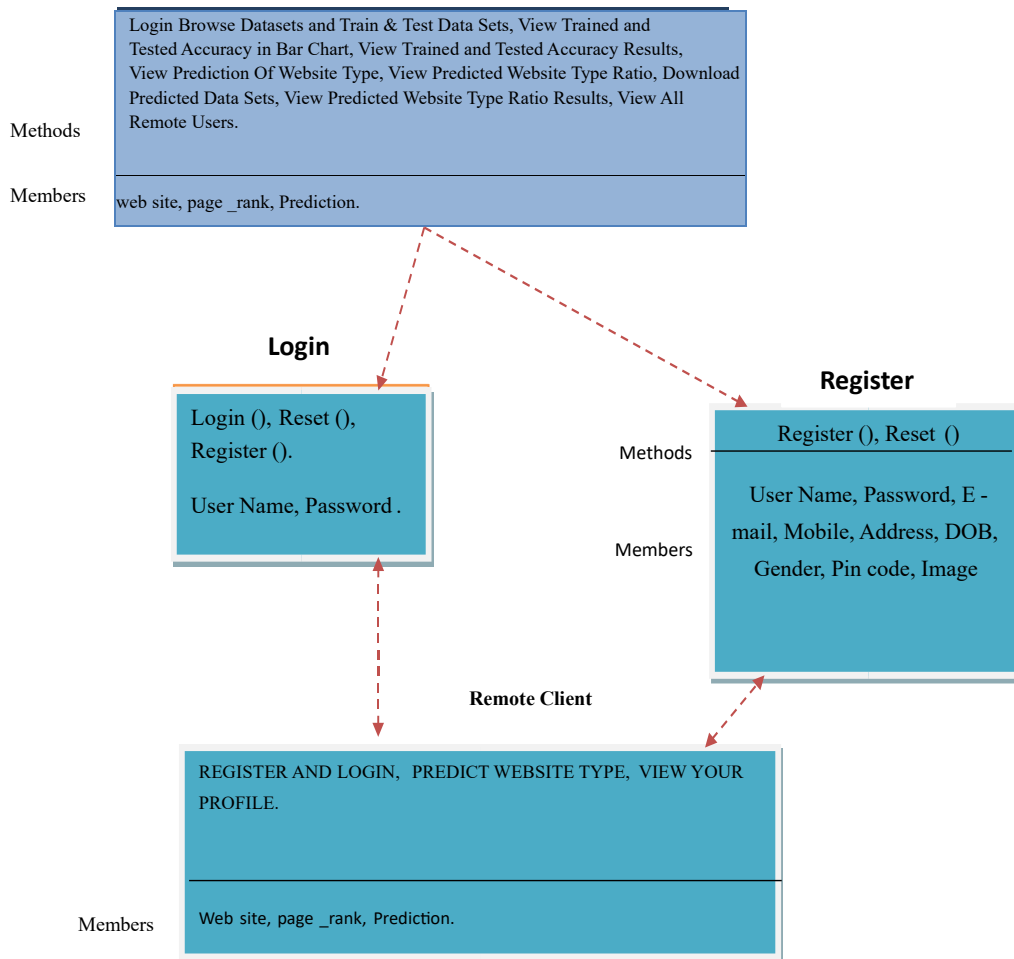


Fig 4.2.2: Analytics Operator

4.2.3 SEQUENCE DIAGRAM:

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.

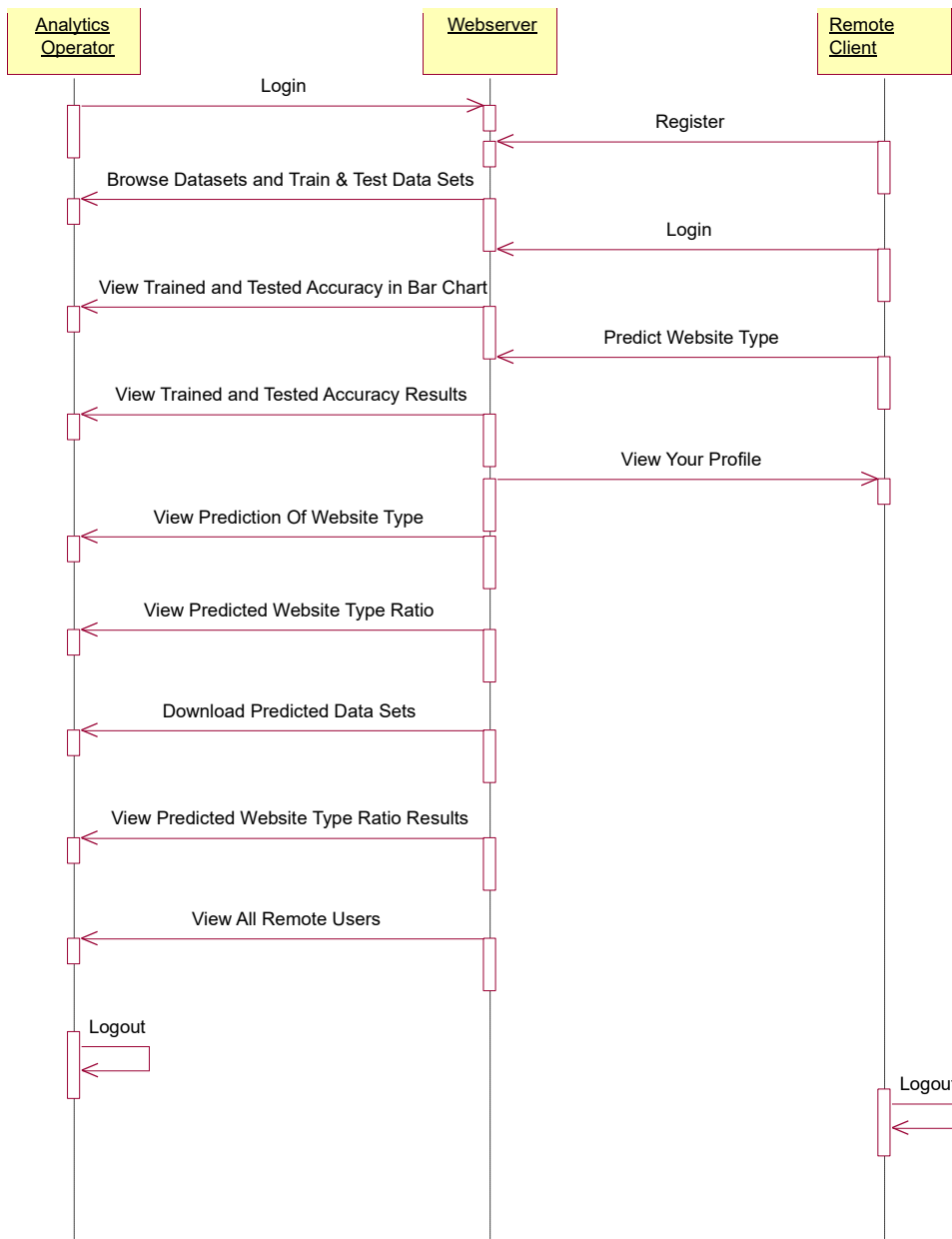


Fig 4.2.3: Sequence Diagram

4.2.4 COLLABRATION DIAGRAM

A collaboration diagram, also known as a communication diagram, is an illustration of the relationships and interactions among software objects in the Unified Modeling Language (UML). Developers can use these diagrams to portray the dynamic behavior of a particular use case and define the role of each object.

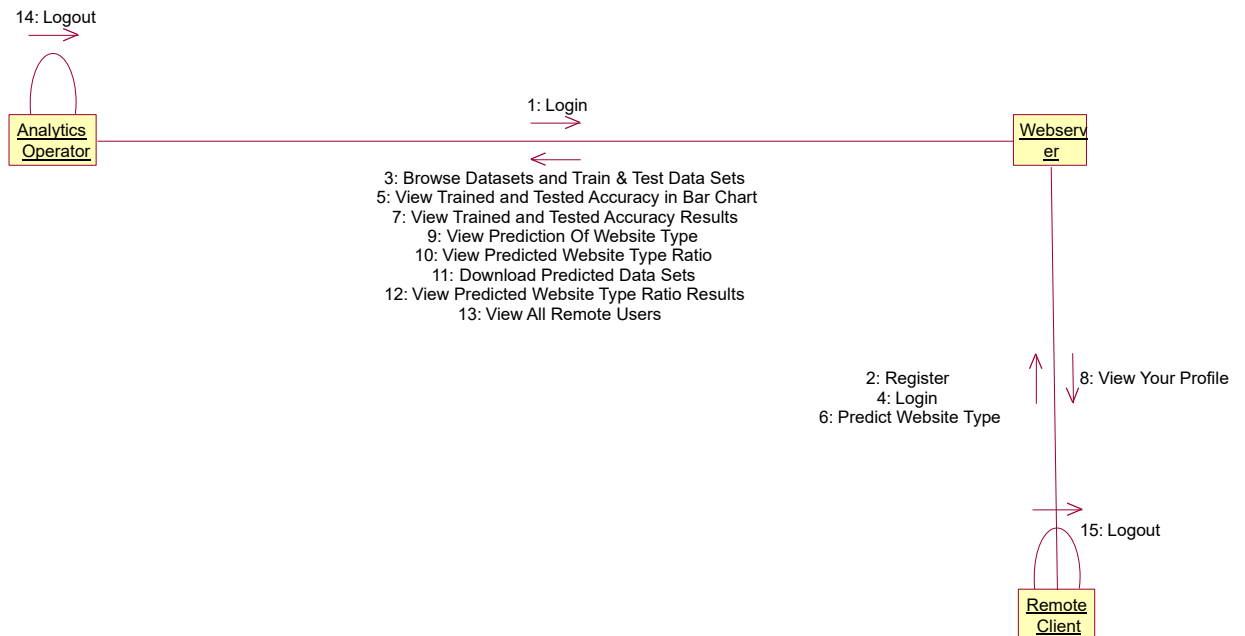


Fig 4.2.4: Collaboration Diagram

4.2.5 ACTIVITY DIAGRAM:

Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control.

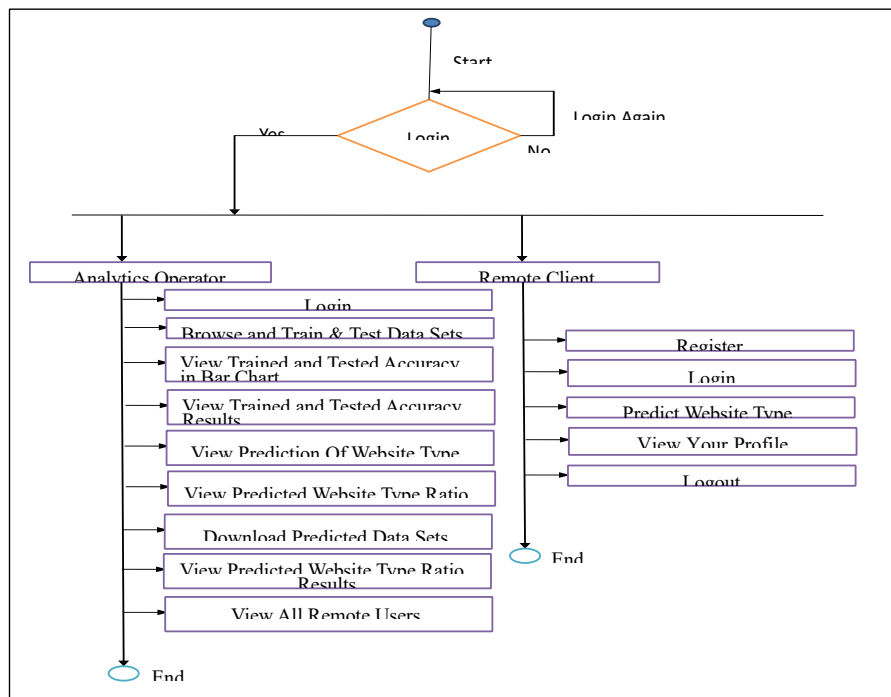


Fig 4.2.5: Activity Diagram

4.2.6 COMPONENT DIAGRAM:

Component diagrams are used in modeling the physical aspects of object-oriented systems that are used for visualizing, specifying, and documenting component-based systems and also for constructing executable systems through forward and reverse engineering. Component diagrams are essentially class diagrams that focus on a system's components that often used to model the static implementation view of a system.

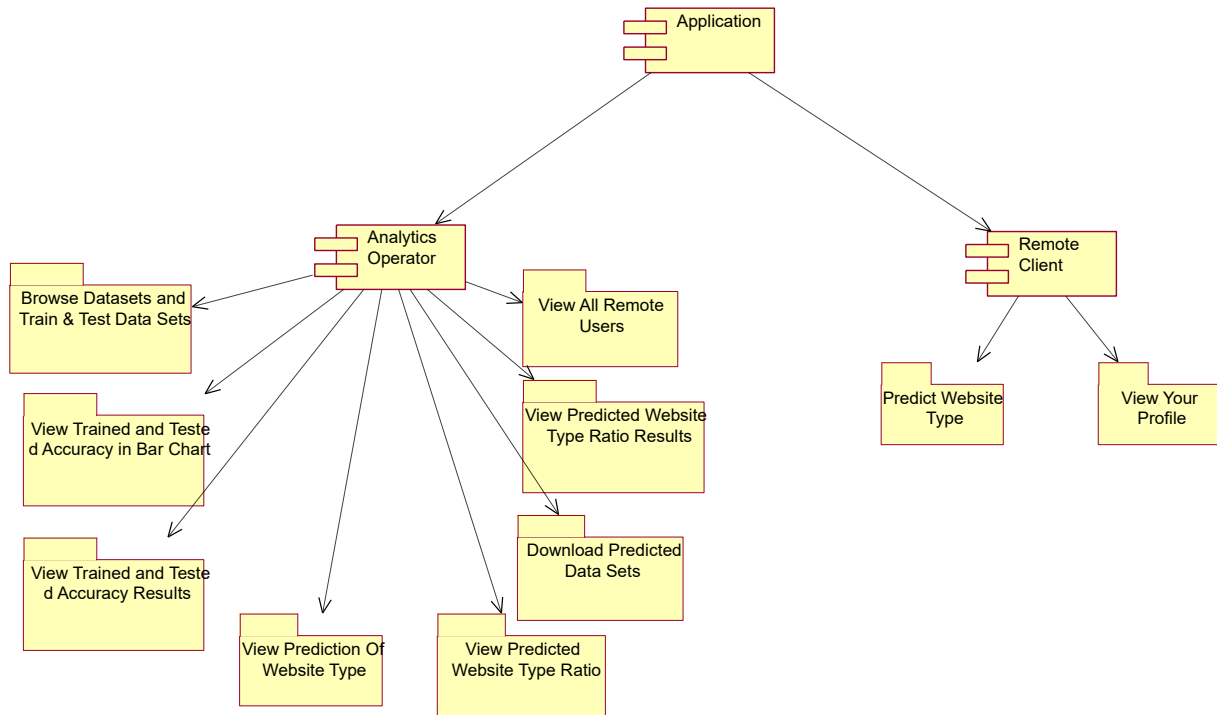


Fig 4.2.6: Component Diagram

4.2.7 DEPLOYMENT DIAGRAM:

Deployment diagrams are used to visualize the topology of the physical components of a system, where the software components are deployed. Deployment diagrams are used to describe the static deployment view of a system. Deployment diagrams consist of nodes and their relationships.

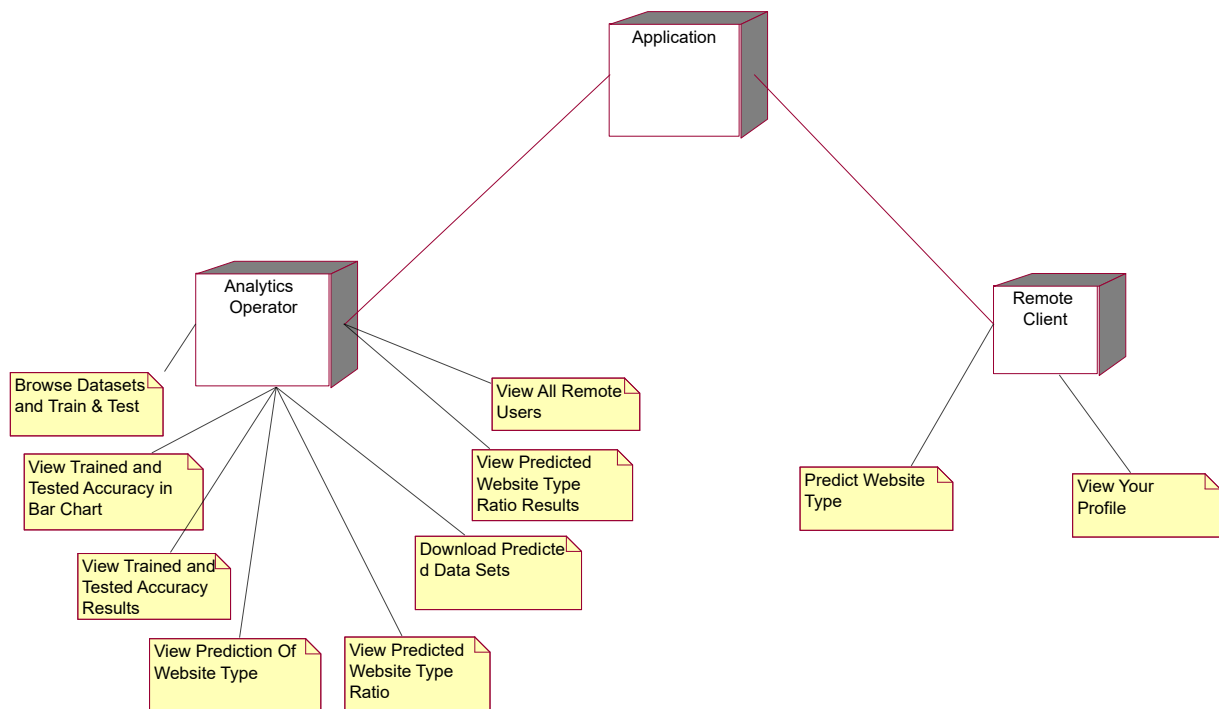


Fig 4.2.7:Deployment Diagram

4.2.8 E-R DIAGRAM

An Entity Relationship Diagram is a diagram that represents relationships among entities in a database. It is commonly known as an ER Diagram. An ER Diagram in DBMS plays a crucial role in designing the database. Today's business world previews all the requirements demanded by the users in the form of an ER Diagram.

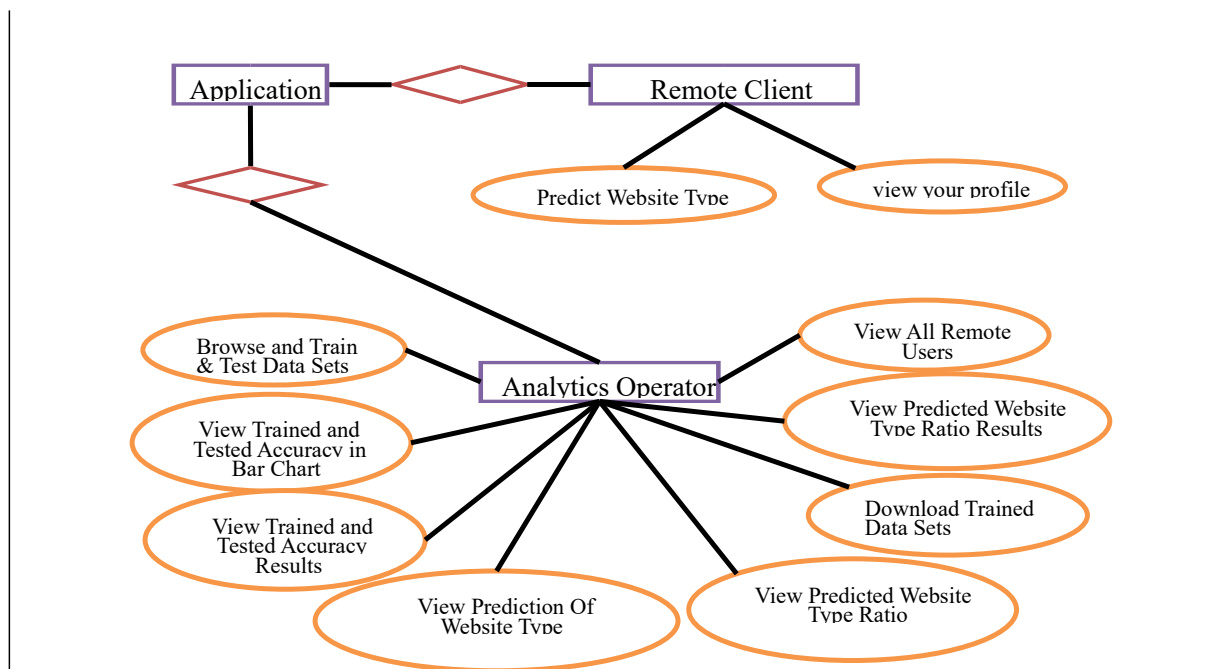


Fig 4.2.8: E-R Diagram

4.2.9 DATA DICTIONARY

Database: phikita

Table name: auth_group

Column	Data Type	Constraints	Description
id	Int(11)	Primary key	Unique Identifier
name	Varchar(1000)	Not null	name

Table Name: Auth_group_Permission

Column	Data Type	Constraints	Description
id	Int(11)	Primary key	Unique Identifier
Group id	Int(11)	Primary Key	Unique Identifier
Permission_id	Int(11)	Primary Key	Unique Identifier

Table Name: auth_permission

Column	Data Type	Constraints	Description
id	Int(11)	Primary key	Unique Identifier
Name	Int(255)	Not Null	name
Content_type_id	Int(11)	Primary Key	Unique Identifier
Code name	Int(100)	Not Null	name

Table Name: auth_User

Column	Data Type	Constraints	Description
id	Int(11)	Primary key	Unique Identifier
Password	varchar(128)	Not Null	password
Last_Login	Datetime(6)	Not Null	Last login
Is_superuser	tinyint(1)	Not Null	Name

username	Varchar(150)	Not Null	Username
lastname	Varchar(30)	Not Null	Lastname
email	Varchar(150)	Not Null	Email id
Is_staff	Tinyint(1)	Not Null	Staff
Is_active	Tinyint(1)	Not Null	Active
Date_joined	Datetime(6)	Not Null	Date and time

Table Name: auth_user_groups

Column	Data Type	Constraints	Description
id	Int(11)	Primary key	Unique Identifier
User_id	Int(11)	Primary Key	Unique Identifier
Group_id	Int(11)	Primary Key	Unique Identifier

CHAPTER-5

IMPLEMENTATION

5.1 MODULE

The major modules of the project are

1. Analytics Operator
2. Remote Client

5.1.1MODULE DESCRIPTION

Service Controller

In this module, the Service Provider has to login by using valid user name and password. After login successful he can do some operations such as Browse Datasets and Train & Test Data Sets, View Trained and Tested Accuracy in Bar Chart, View Trained and Tested Accuracy Results, View Prediction Of Website Type, View Predicted Website Type Ratio, Download Predicted Data Sets, View Predicted Website Type Ratio Results, View All Remote Users.

Remote Access User

In this module, there are n numbers of users are present. User should register before doing any operations. Once user registers, their details will be stored to the database. After registration successful, he has to login by using authorized user name and password.

Once Login is successful user will do some operations like Register And Login, Predict Traffic Type, View Your Profile.

5.2 TECHNOLOGY

5.2.1 PYTHON

Python is a high-level, interpreted, interactive and object-oriented scripting language. Python is designed to be highly readable. It uses English keywords frequently where as other languages use punctuation, and it has fewer syntactical constructions than other languages.

- Python is Interpreted , Interactive ,Object-Oriented, Beginner's Language

History of Python

- Python was developed by Guido van Rossum in the late eighties and early nineties at the National Research Institute for Mathematics and Computer Science in the Netherlands.
- Python is derived from many other languages, including ABC, Modula-3, C, C++, Algol-68, SmallTalk, and Unix shell and other scripting languages.

Python Features

Python's features include: Easy-to-learn, Easy-to-read, Easy-to-maintain, A broad standard library, Interactive Mode, Portable, Extendable Databases, GUI Programming, Scalable

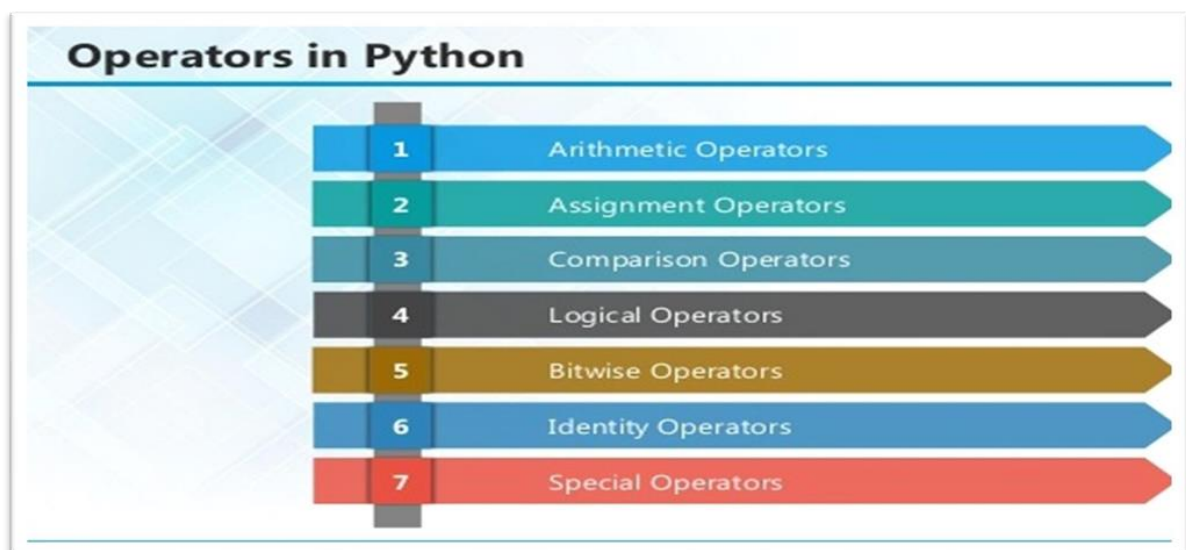


Fig 5.2.1 Python Operators

ARITHMETIC OPERATORS

Operator	Description
+ Addition	Adds values on either side of the operator.
- Subtraction	Subtracts right hand operand from left hand operand.
* Multiplication	Multiplies values on either side of the operator
/ Division	Divides left hand operand by right hand operand
% Modulus	Divides left hand operand by right hand operand and returns remainder
** Exponent	Performs exponential (power) calculation on operators

//	The division of operands where the result is the quotient in which the digits after the decimal point are removed.
----	--

ASSIGNMENT OPERATOR

Operator	Description
=	Assigns values from right side operands to left side operand
+= Add AND	It adds right operand to the left operand and assign the result to left operand
-= Subtract AND	It subtracts right operand from the left operand and assign the result to left operand
*= Multiply AND	It multiplies right operand with the left operand and assign the result to left operand
/= Divide AND	It divides left operand with the right operand and assign the result to left operand

IDENTITY OPERATOR

Operator	Description
is	Evaluates to true if the variables on either side of the operator point to the same object and false otherwise.
is not	Evaluates to false if the variables on either side of the operator point to the same object and true otherwise.

COMPARISON OPERATOR

Operator	Description
& Binary AND	Operator copies a bit to the result if it exists in both operands
Binary OR	It copies a bit if it exists in either operand.
^ Binary XOR	It copies the bit if it is set in one operand but not both.

~ Binary Ones Complement	It is unary and has the effect of 'flipping' bits.
<< Binary Left Shift	The left operands value is moved left by the number of bits specified by the right operand.
>> Binary Right Shift	The left operands value is moved right by the number of bits specified by the right operand.

LOGICAL OPERATOR

Operator	Description
and Logical AND	If both the operands are true then condition becomes true.
or Logical OR	If any of the two operands are non-zero then condition becomes true.
not Logical NOT	Used to reverse the logical state of its operand.

Precedence Operator

Operator	Description
**	Exponentiation (raise to the power)
~ + -	Complement, unary plus and minus (method names for the last two are +@ and -@)
* / % //	Multiply, divide, modulo and floor division
+ -	Addition and subtraction
>> <<	Right and left bitwise shift
&	Bitwise 'AND'
^	Bitwise exclusive 'OR' and regular 'OR'
<= < > >=	Comparison operators

<> == !=	Equality operators
= %= /= //= -= += *= **=	Assignment operators
is is not	Identity operators
in not in	Membership operators
not or and	Logical operators

LIST

The list is a most versatile data type available in Python which can be written as a list of comma-separated values (items) between square brackets. Important thing about a list is that items in a list need not be of the same type.

Built-in List Functions & Methods:

Python includes the following list functions –

SN	Function with Description
1	<u>cmp(list1, list2)</u> Compares elements of both lists.
2	<u>len(list)</u> Gives the total length of the list.
3	<u>max(list)</u> Returns item from the list with max value.
4	<u>min(list)</u> Returns item from the list with min value.
5	<u>list(seq)</u> Converts a tuple into list.

Python includes following list methods

SN	Methods with Description
1	<u>list.append(obj)</u> Appends object obj to list
2	<u>list.count(obj)</u> Returns count of how many times obj occurs in list
3	<u>list.extend(seq)</u> Appends the contents of seq to list
4	<u>list.index(obj)</u> Returns the lowest index in list that obj appears
5	<u>list.insert(index, obj)</u> Inserts object obj into list at offset index
6	<u>list.pop(obj=list[-1])</u> Removes and returns last object or obj from list
7	<u>list.remove(obj)</u> Removes object obj from list
8	<u>list.reverse()</u> Reverses objects of list in place
9	<u>list.sort([func])</u> Sorts objects of list, use compare function if given

TUPLES

A tuple is a sequence of immutable Python objects. Tuples are sequences, just like lists. The differences between tuples and lists are, the tuples cannot be changed unlike lists and tuples use parentheses, whereas lists use square brackets.

• Accessing Values in Tuples:

To access values in tuple, use the square brackets for slicing along with the index or indices to obtain value available at that index

Updating Tuples:

Tuples are immutable which means you cannot update or change the values of tuple elements. We are able to take portions of existing tuples to create new tuples as the following example demonstrates –

Delete Tuple Elements

Removing individual tuple elements is not possible. There is, of course, nothing wrong with putting together another tuple with the undesired elements discarded.

Built-in Tuple Functions

SN	Function with Description
1	cmp(tuple1, tuple2) : Compares elements of both tuples.
2	len(tuple) : Gives the total length of the tuple.
3	max(tuple) : Returns item from the tuple with max value.
4	min(tuple) : Returns item from the tuple with min value.
5	tuple(seq) : Converts a list into tuple.

DICTIONARY

Each key is separated from its value by a colon (:), the items are separated by commas, and the whole thing is enclosed in curly braces. An empty dictionary without any items is written with just two curly braces, like this: {}. Keys are unique within a dictionary while values may not be. Keys are unique within a dictionary while values may not be.

Accessing Values in Dictionary:

To access dictionary elements, you can use the familiar square brackets along with the key to obtain its value. Following is a simple example –

```
dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}  
print "dict['Name']: ", dict['Name']  
print "dict['Age']: ", dict['Age']
```

Result –

```
dict['Name']: Zara  
dict['Age']: 7
```

Built-in Dictionary Functions & Methods –

Python includes the following dictionary functions –

SN	Function with Description
1	<u>cmp(dict1, dict2)</u> Compares elements of both dict.

2	<u>len(dict)</u> Gives the total length of the dictionary. This would be equal to the number of items in the dictionary.
3	<u>str(dict)</u> Produces a printable string representation of a dictionary
4	<u>type(variable)</u> Returns the type of the passed variable. If passed variable is dictionary, then it would return a dictionary type.

Python includes following dictionary methods –

SN	Methods with Description
1	dict.clear() :Removes all elements of dictionary <i>dict</i>
2	dict. Copy() :Returns a shallow copy of dictionary <i>dict</i>
3	dict.fromkeys() :Create a new dictionary with keys from seq and values <i>set</i> to <i>value</i> .
4	dict.get(key, default=None) :For <i>key</i> key, returns value or default if key not in dictionary
5	dict.has_key(key) :Returns <i>true</i> if key in dictionary <i>dict</i> , <i>false</i> otherwise
6	dict.items() :Returns a list of <i>dict</i> 's (key, value) tuple pairs
7	dict.keys() :Returns list of dictionary <i>dict</i> 's keys
8	dict.setdefault(key, default=None) :Similar to get(), but will set dict[key]=default if <i>key</i> is not already in dict
9	dict.update(dict2) :Adds dictionary <i>dict2</i> 's key-values pairs to <i>dict</i>
10	dict.values() :Returns list of dictionary <i>dict</i> 's values

Defining a Function

Simple rules to define a function in Python.

- Function blocks begin with the keyword `def` followed by the function name and parentheses ().

- Any input parameters or arguments should be placed within these parentheses. You can also define parameters inside these parentheses.
- The first statement of a function can be an optional statement - the documentation string of the function or *docstring*.
- The code block within every function starts with a colon (:) and is indented.
- The statement `return [expression]` exits a function, optionally passing back an expression to the caller. A return statement with no arguments is the same as `return None`.

```
def functionname ( parameters ) :  
  
    "function_docstring"  
    function_suite  
    return [expression]
```

Calling a Function

Defining a function only gives it a name, specifies the parameters that are to be included in the function and structures the blocks of code. Once the basic structure of a function is finalized, you can execute it by calling it from another function or directly from the Python prompt. When the above code is executed,

Scope of Variables

All variables in a program may not be accessible at all locations in that program. This depends on where you have declared a variable.

Global variables

Local variables

Global vs. Local variables

Variables that are defined inside a function body have a local scope, and those defined outside have a global scope.

Example:

The Python code for a module named `aname` normally resides in a file named `aname.py`. Here's an example of a simple module, `support.py`

```
def print_func ( par ) :  
  
    print "Hello : ", par
```

```
return
```

The import Statement

The import has the following syntax:

```
import module1 [, module2 [, ... moduleN]
```

When the interpreter encounters an import statement, it imports the module if the module is present in the search path. A search path is a list of directories that the interpreter searches before importing a module. A module is loaded only once, regardless of the number of times it is imported.

Packages in Python

A package is a hierarchical file directory structure that defines a single Python application environment

Printing to the Screen

The simplest way to produce output is using the print statement where you can pass zero or more expressions separated by commas.

```
print "Python is really a great language,", "isn't it?"
```

Result:

```
Python is really a great language, isn't it?
```

Reading Keyboard Input

Python provides two built-in functions to read a line of text from standard input, which by default comes

from the keyboard. These functions are –

- raw_input
- input

The raw_input([prompt]) function reads one line from standard input and returns it as a string (removing the trailing newline).

The input([prompt]) function is equivalent to raw_input, except that it assumes the input is a valid Python expression and returns the evaluated result to you.

Opening and Closing Files

Python provides basic functions and methods necessary to manipulate files by default. You can do most of the file manipulation using a file object.

The open Function

Before you can read or write a file, you have to open it using Python's built-in `open()` function. This function creates a file object, which would be utilized to call other support methods associated with it.

Syntax:

```
file object = open(file_name [, access_mode] [, buffering])
```

Here are parameter details:

- **file_name:** The `file_name` argument is a string value that contains the name of the file that you want to access.

Modes	Description
r	Opens a file for reading only.
rb	Opens a file for reading only in binary format
r+	Opens a file for both reading and writing.
rb+	Opens a file for both reading and writing in binary format.
w	Opens a file for writing only. Overwrites the file if the file exists.
wb	Opens a file for writing only in binary format. Overwrites the file if the file exists.
w+	Opens a file for both writing and reading. Overwrites the existing file if the file exists.
wb+	Opens a file for both writing and reading in binary format. Overwrites the existing file if the file exists.
a	Opens a file for appending. The file pointer is at the end of the file if the file exists
ab	Opens a file for appending in binary format. The file pointer is at the end of the file if the file exists.
a+	Opens a file for both appending and reading. The file pointer is at the end of the file if the file exists

- **access_mode:** The `access_mode` determines the mode in which the file has to be opened, i.e., read, write, append, etc. A complete list of possible values is given below in the table. This is optional parameter and the default file access mode is read (r).
- **buffering:** If the buffering value is set to 0, no buffering takes place. If the buffering value is 1, line buffering is performed while accessing a file. If you specify the buffering value as an integer greater than 1, then buffering action is performed with the indicated buffer size. If negative, the buffer size is the system default(default behavior).

Here is a list of the different modes of opening a file –

The file Object Attributes

Once a file is opened and you have one file object, you can get various information related to that file

Here is a list of all attributes related to file object:

Attribute	Description
<code>file.closed</code>	Returns true if file is closed, false otherwise.
<code>file.mode</code>	Returns access mode with which file was opened.
<code>file.name</code>	Returns name of the file.
<code>file.softspace</code>	Returns false if space explicitly required with print, true otherwise.

Directories in Python

All files are contained within various directories, and Python has no problem handling these too.

The **os** module has several methods that help you create, remove, and change directories.

The `mkdir()` You can use the `mkdir()` method of the `os` module to create directories in the current directory. You need to supply an argument to this method which contains the name of the directory to be created.

Syntax

```
os.mkdir("newdir")
```

Example

Following is the example to create a directory *test* in the current directory –

```
#!/usr/bin/python
import os
# Create a directory "test"
```

```
os.mkdir("test")
```

The chdir() Method

You can use the chdir() method to change the current directory. The chdir() method takes an argument, which is the name of the directory that you want to make the current directory.

Syntax

```
os.chdir("newdir")
```

Example

Following is the example to go into "/home/newdir" directory –

```
#!/usr/bin/python import
os
# Changing a directory to "/home/newdir"
os.chdir("/home/newdir")
```

The getcwd() method displays the current working directory.

Syntax

```
os.getcwd()
```

Following is the example to give current directory –

```
import os
# This would give location of the current directory os.getcwd()
```

The rmdir() method deletes the directory, which is passed as an argument in the method.

Before removing a directory, all the contents in it should be removed.

Syntax:

```
os.rmdir('dirname')
```

Following is the example to remove "/tmp/test" directory. It is required to give fully qualified name of the directory, otherwise it would search for that directory in the current directory.

```
import os
# This would remove "/tmp/test" directory. os.rmdir(
"/tmp/test" )
```

EXCEPTION NAME	DESCRIPTION
Exception	Base class for all exceptions
StopIteration	Raised when the next() method of an iterator does not point to any object.
SystemExit	Raised by the sys.exit() function.
StandardError	Base class for all built-in exceptions except StopIteration and SystemExit.
ArithmeticError	Base class for all errors that occur for numeric calculation.
OverflowError	Raised when a calculation exceeds maximum limit for a numeric type.
FloatingPointError	Raised when a floating point calculation fails.
ZeroDivisionError	Raised when division or modulo by zero takes place for all numeric types.
AssertionError	Raised in case of failure of the Assert statement.
AttributeError	Raised in case of failure of attribute reference or assignment.
EOFError	Raised when there is no input from either the raw_input() or input() function and the end of file is reached.
KeyboardInterrupt	Raised when the user interrupts program execution, usually by pressing Ctrl+c.
LookupError	Base class for all lookup errors.
IndexError	Raised when an index is not found in a sequence.
KeyError	Raised when the specified key is not found in the dictionary.
NameError	Raised when an identifier is not found in the local or global namespace.

UnboundLocalError	Raised when trying to access a local variable in a function or method but no value has been assigned to it.
EnvironmentError	Base class for all exceptions that occur outside the Python environment.
IOError	Raised when an input/ output operation fails, such as the print statement or the open() function when trying to open a file that does not exist.
IOError	Raised for operating system-related errors.
SyntaxError	Raised when there is an error in Python syntax.
IndentationError	Raised when indentation is not specified properly.
SystemError	Raised when the interpreter finds an internal problem, but
	when this error is encountered the Python interpreter does not exit.
SystemExit	Raised when Python interpreter is quit by using the sys.exit() function. If not handled in the code, causes the interpreter to exit.
TypeError	Raised when an operation or function is attempted that is invalid for the specified data type.
ValueError	Raised when the built-in function for a data type has the valid type of arguments, but the arguments have invalid values specified.
RuntimeError	Raised when a generated error does not fall into any category.
NotImplementedError	Raised when an abstract method that needs to be implemented in an inherited class is not actually implemented.

File & Directory Related Methods

There are three important sources, which provide a wide range of utility methods to handle and manipulate files & directories on Windows and Unix operating systems. They are as follows

—

- File Object Methods: The file object provides functions to manipulate files.
- OS Object Methods: This provides methods to process files as well as directories.

Python provides two very important features to handle any unexpected error in your Python programs and to add debugging capabilities in them –

- **Exception Handling:** This would be covered in this tutorial. Here is a list standard Exceptions available in Python: `Standard_Exceptions`.
- **Assertions:** This would be covered in `Assertions_in_Python`

List of Standard Exceptions – What is Exception?

An exception is an event, which occurs during the execution of a program that disrupts the normal flow of the program's instructions. In general, when a Python script encounters a situation that it cannot cope with, it raises an exception. An exception is a Python object that represents an error.

When a Python script raises an exception, it must either handle the exception immediately otherwise it terminates and quits.

Handling an exception

If you have some suspicious code that may raise an exception, you can defend your program by placing the suspicious code in a **try:** block. After the **try:** block, include an **except:** statement, followed by a block of code which handles the problem as elegantly as possible.

The Python standard for database interfaces is the Python DB-API. Most Python database interfaces adhere to this standard.

You can choose the right database for your application. Python Database API supports a wide range of

database servers such as –

- GadFly
- MySQL
- PostgreSQL
- Microsoft SQL Server 2000
- Informix
- Interbase
- Oracle

- Sybase

The DB API provides a minimal standard for working with databases using Python structures and syntax wherever possible. This API includes the following:

- Importing the API module.
- Acquiring a connection with the database.
- Issuing SQL statements and stored procedures.
- Closing the connection

5.3 SYSTEM TESTING

What do you mean by software testing?

Testing involves operation of a system or application under controlled conditions and evaluating the results. The controlled conditions should include both normal and abnormal conditions. Testing should intentionally attempt to make things go wrong to determine if things happen when they shouldn't or things don't happen when they should. It is oriented to 'detection'.

Unit Testing

Unit testing is a software development process in which the smallest testable parts of an application, called units, are individually and independently scrutinized for proper operation. Unit testing is often automated but it can also be done manually. This testing mode is a component of Extreme Programming (XP), a pragmatic method of software development that takes a meticulous approach to building a product by means of continual testing and revision.

Unit tests are written from a programmer's perspective. They ensure that a particular method of a class successfully performs a set of specific tasks. Each test confirms that a method produces the expected output when given a known input.

Performance Testing:

Performance testing is the process of determining the speed or effectiveness of a computer, network, software program or device. This process can involve quantitative tests done in a lab, such as measuring the response time or the number of MIPS (millions of instructions per second) at which a system functions. Qualitative attributes such as Reliability, scalability and interoperability may also be evaluated. Performance testing is often done in conjunction with stress testing.

Performance testing can verify that a system meets the specifications claimed by its manufacturer or vendor. The process can compare two or more devices or programs in terms of parameters such as speed, data transfer rate, bandwidth, throughput, efficiency or reliability.

Performance testing can also be used as a diagnostic aid in locating communications bottlenecks. Often a system will work much better if a problem is resolved at a single point or in a single component. For example, even the fastest computer will function poorly on today's Web if the connection occurs at only 40 to 50 Kbps (kilobits per second).

Integration Testing:

Integration testing, also known as integration and testing (I&T), is a software development process which program units are combined and tested as groups in multiple ways. In this context, a unit is defined as the smallest testable part of an application. Integration testing can expose problems with the interfaces among program components before trouble occurs in real-world program execution. Integration testing is a component of Extreme Programming (XP), a pragmatic method of software development that takes a meticulous approach to building a product by means of continual testing and revision.

Test cases:

Test case1:

Test case for Login form:

FUNCTION:	LOGIN
EXPECTED RESULTS:	Should Validate the user and check his existence in database
ACTUAL RESULTS:	Validate the user and checking the user against the database
LOW PRIORITY	No
HIGH PRIORITY	Yes

Test case2:

Test case for Remote Access User Registration Form

FUNCTION:	USER REGISTRATION
EXPECTED RESULTS:	Should check if all the fields are filled by the user and saving the user to database.

ACTUAL RESULTS:	Checking whether all the fields are filled by user or not through validations and saving user.
LOW PRIORITY	No
HIGH PRIORITY	Yes

Test case3:

Test case for Change Password:

When the old password does not match with the new password ,then this results in displaying an error message as “ OLD PASSWORD DOES NOT MATCH WITH THE NEW PASSWORD”.

FUNCTION:	Change Password
EXPECTED RESULTS:	Should check if old password and new password fields are filled by the user and saving the user to database.
ACTUAL RESULTS:	Checking whether all the fields are filled by user or not through validations and saving user.
LOW PRIORITY	No
HIGH PRIORITY	Yes

Test case 4:

Test case for Forget Password:

When a user forgets his password he is asked to enter Login name, ZIP code, Mobile number. If these are matched with the already stored ones then user will get his Original password.

Module	Functionality	Test Case	Expected Results	Actual Results	Result	Priority
er	Login Usecase	Navigate To Www.Sample.Com Click On Submit Button Without	A Validation Should Be As Below “Please	A Validation Has Been Populated As Expected	Pass	High

		Entering Username and Password	Enter Valid Username & Password”			
--	--	--------------------------------	----------------------------------	--	--	--

		Navigate To Www.Sample.Com Click On Submit Button With Out Filling Password And With Valid Username.Test Username Field .Navigate To Www.Sample.Com Enter Both Username And Password Wrong And Hit Enter	A Validation Should Be As Below “Please Enter Valid Password Or Password Field Can Not Be Empty “ A Validation Shown As Below “The Username Entered Is Wrong”	A Validation Is Shown As Expected A Validation Is Shown As Expected	Pass	High
--	--	--	--	--	------	------

		Navigate To Www.Sample.Com Enter Validate Username And Password And Click On Submit	Validate Username And Password In DataBase And Once If They Correct Then Show Main Page	Main Page/ Home Page Has Been Displayed	Pass	High
--	--	---	---	---	------	------

CHAPTER-6

RESULTS&DISCUSSIONS

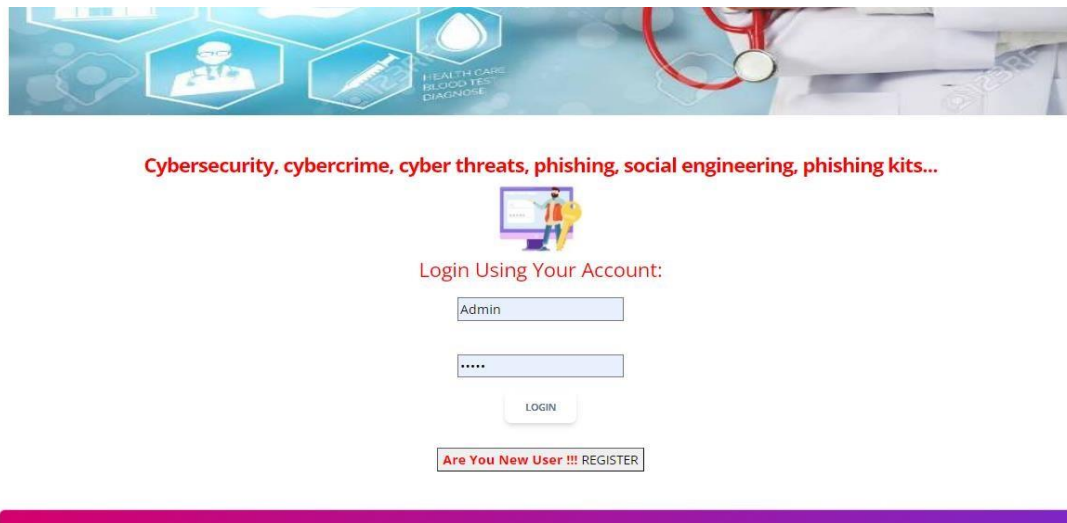


Fig 6.1: Remote Client Login



Fig 6.2: Analytics Operator Login



Fig 6.3: Analytics Operator Home



Fig 6.4: Analytics Operator Operation Browse Dataset, Train and test dataset

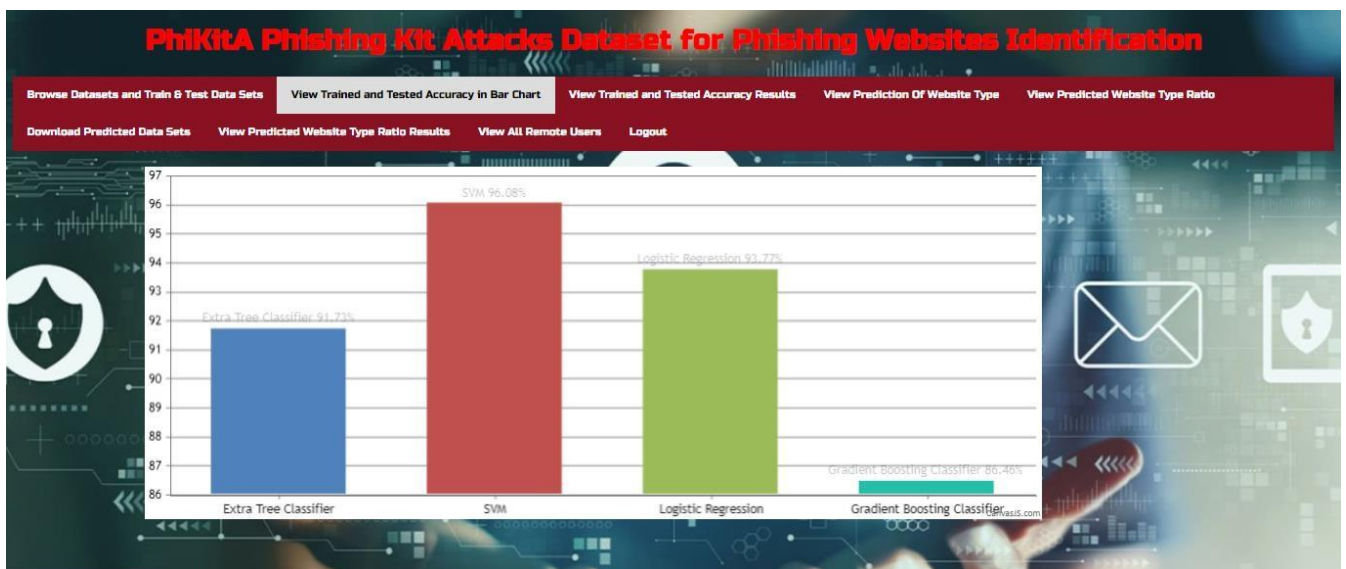


Fig 6.5: Analytics Operator Operation View Trained and Tested Accuracy in Bar Chart



Fig 6.6: Analytics Operator Operation View Trained and Tested Accuracy Results

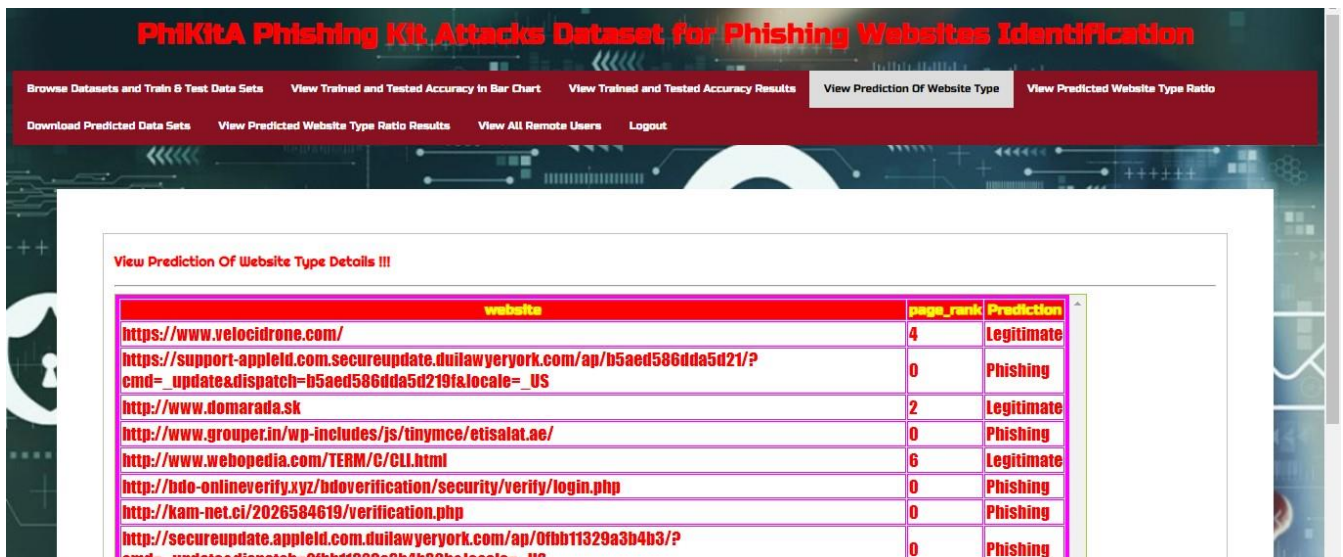


Fig6.7: Analytics Operator Operation View Prediction Of Whether Type



Fig 6.8: Analytics Operator Operation View Predicted whether type Ratio

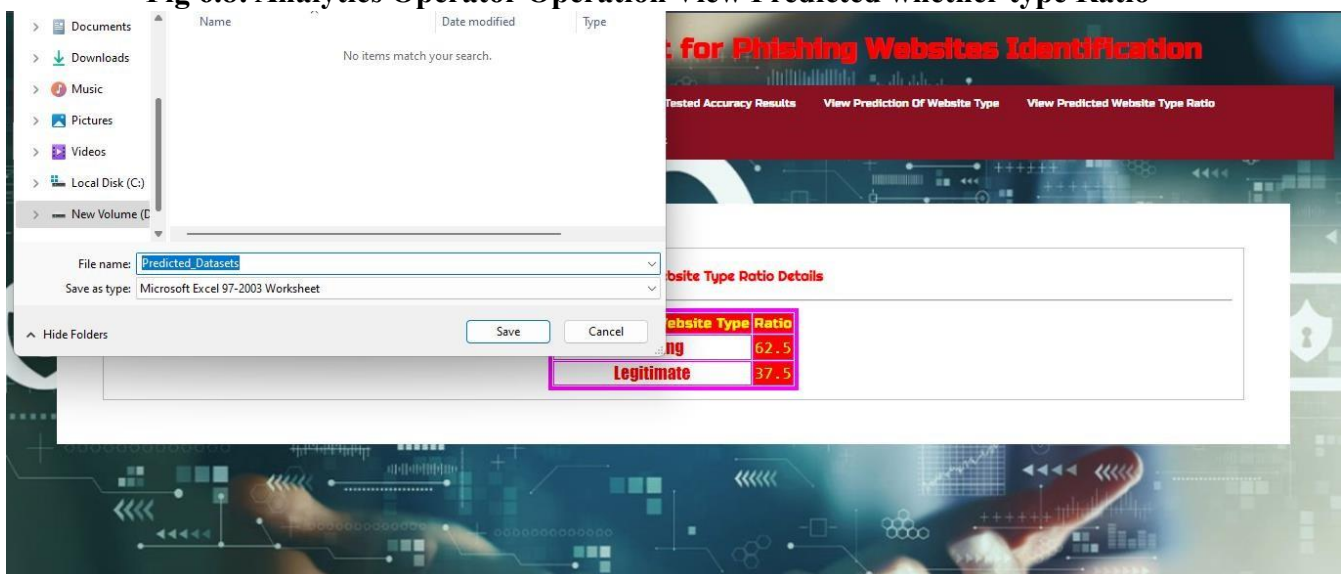


Fig 6.9: Analytics Operator Operation Download Predicted Dataset

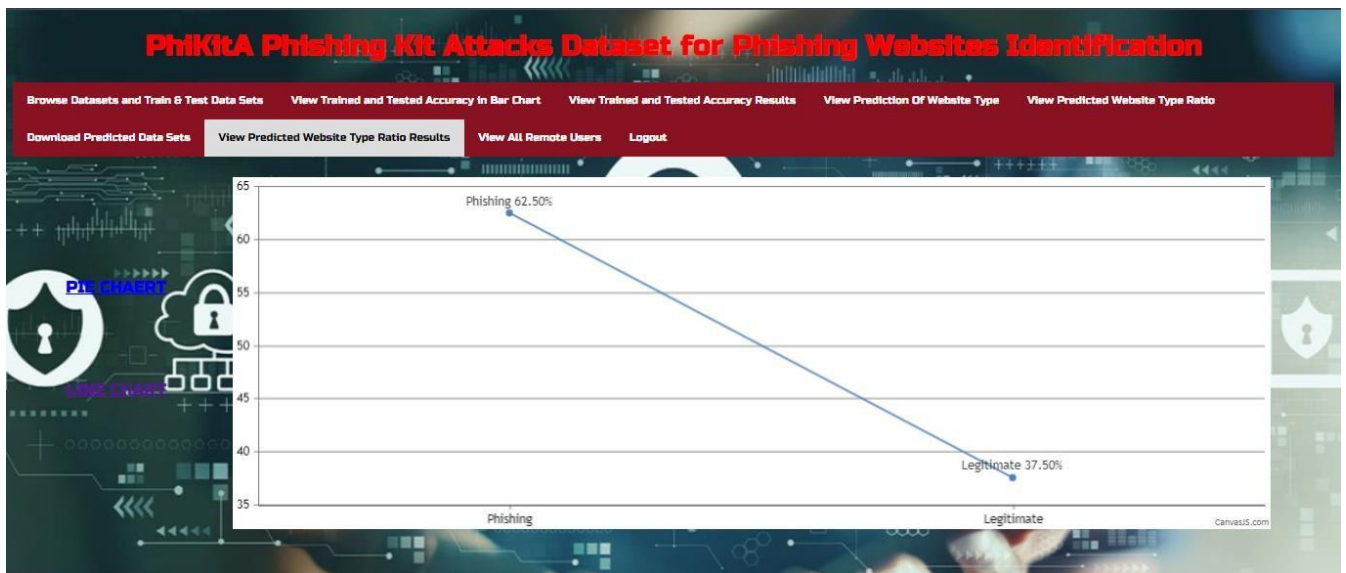


Fig 6.10: Analytics Operator Operation View Predicted Whether type ratio Results

The figure displays a table titled "VIEW ALL REMOTE USERS !!!". The table contains the following data:

USER NAME	EMAIL	Gender	Address	Mob No	Country	State	City
RaghuNath	RaghuNath123@gmail.com	Male	#9028,4th Cross,mallesham	9535866270	India	Karnataka	Bangalore
Manjunath	tmsmanju14@gmail.com	Male	#8928,4th Cross,Vijayanagar	9535866270	India	karnataka	Bangalore

Fig 6.11: Analytics Operator operation View All Remote Clients

CHAPTER-7

CONCLUSION

In this paper, we explore how phishing kit information can be used to support the identification of phishing websites. For this purpose, we proposed a novel methodology to collect data where we have crawled phishing kits and phishing websites generated with the related kit. Following our methodology, we proposed and made publicly available PhiKitA, a dataset containing 510 phishing kits, 859 phishing website samples, and 141 legitimate websites. With PhiKitA, we release a ground truth where researchers can evaluate their proposals for phishing kit analysis, phishing binary classification and multiclass classification experiments.

In this paper, we evaluated three phishing classification and clustering algorithms from the literature and tested them in three experiments in PhiKitA. First, the familiarity experiment showed two phishing kit families: one with the same functionalities and file distribution but different targets, and the other with the same target but different functionalities, ranging from the simplest to the most complex sample. This may indicate that phishing kit programmers design their products to be attractive to phishers and price them according to the range of functions they offer.

After analyzing PhiKitA and its familiarity, we found a family containing seven phishing kits related to 172 different phishing attacks, whose target was Standard Bank. Phishing websites related to this family represent 20% of all phishing website samples in the dataset, which could indicate that PhiKitA might also contain a phishing campaign against Standard Bank.

Of the three algorithms evaluated for phishing detection, the Graph Representation algorithm achieved the highest performance, with an accuracy of 92.50% in PhiKitA. Although this performance is lower than other state-of-the-art approaches in phishing classification, the results show that the information obtained from phishing kits helps to determine whether a sample is phishing. This could be used as supporting information in other approaches.

Finally, in the last experiment, the multi-class classification, we found that the MD5 hash algorithm shows the best performance, with 39.54% of the F1-score. Due to the increasing number of phishing attacks and the use of phishing kits for their deployment, it is worth exploring how to cluster these deployments according to the source of the phishing kit. This clustering could be helpful for both binary classifications and identifying common targets of phishing attacks and phishing campaigns.

FUTURE SCOPE

Future research on the PhiKitA dataset should focus on several key areas to enhance its utility and effectiveness. First, expanding the dataset to include a broader and more recent variety of phishing kits and website samples will improve its robustness. Advanced phishing detection algorithms, particularly those leveraging machine learning and deep learning, should be developed to better utilize phishing kit data. Implementing real-time data collection and processing will enable prompt identification of new phishing campaigns. Improving multi-class classification algorithms beyond the current MD5 hash performance (39.54% F1-score) is essential for more accurate results.

Additionally, clustering techniques should be refined to better identify common targets and phishing campaigns, with a focus on temporal and spatial analysis. Cross-dataset validation is crucial to ensure the generalizability of detection models. Studying user behavior with phishing websites can provide insights for better user education and awareness programs. Finally, integrating PhiKitA data with existing cybersecurity tools will enhance real-time protection and alerts, making phishing detection more effective and adaptive to evolving threats.

REFERENCES

- [1] I. T. Union. (2021). Measuring Digital Development: Facts and Figures. [Online]. Available:
<https://www.itu.int/en/ITU-D/Statistics/Documents/facts/FactsFigures2021.pdf>
- [2] R. M. A. Mohammad, “A lifelong spam emails classification model,” *Appl. Comput. Informat.*, Jul. 2020.[Online].
Available:
<https://www.emerald.com/insight/content/doi/10.1016/j.aci.2020.01.002/full/html>
- [3] F. Jáñez-Martino, E. Fidalgo, S. González-Martínez, and J. Velasco-Mata, “Classification of spam emails through hierarchical clustering and supervised learning,” 2020, arXiv:2005.08773.
- [4] J. Velasco-Mata, V. Gonzalez-Castro, E. F. Fernandez, and E. Alegre, “Efficient detection of botnet traffic by features selection and decision trees,” *IEEE Access*, vol. 9, pp. 120567–120579, 2021.
- [5] A. Mihoub, O. B. Fredj, O. Cheikhrouhou, A. Derhab, and M. Krichen, “Denial of service attack detection and mitigation for Internet of Things using looking-back-enabled machine learning techniques,” *Comput. Electr. Eng.*, vol. 98, Mar. 2022, Art. no. 107716, doi: 10.1016/j.compeleceng.2022.107716.
- [6] E. Fidalgo, E. Alegre, L. Fernández-Robles, and V. González-Castro, “Classifying suspicious content in Tor darknet through semantic attention keypoint filtering,” *Digit. Invest.*, vol. 30, pp. 12–22, Sep. 2019. [Online]. Available:
<http://www.sciencedirect.com/science/article/pii/S1742287619300027> P. Blanco-Medina, E. Fidalgo, E. Alegre, and F. Janez-Martino, “Improving text recognition in Tor darknet with rectification and super-resolution techniques,” in *Proc. 9th Int. Conf. Imag. Crime Detection Prevention (ICDP)*, 2019, pp. 32–37.
- [7] E. Figueras-Martín, R. Magán-Carrión, and J. Boubeta-Puig, “Drawing the web structure and content analysis beyond the tor darknet: Freenet as a case of study,” *J. Inf. Secur. Appl.*, vol. 68, Aug. 2022, Art. no. 103229, doi: 10.1016/j.jisa.2022.103229.
- [8] C. A. Murty, H. Rana, R. Verma, R. Pathak, and P. H. Rughani, “Building an AI/ML based classification framework for dark web text data,” in *Proc. Int. Conf. Comput. Commun. Netw. Cham, Switzerland: Springer*, 2022, pp. 93–111.
- [9] D. Chaves, E. Fidalgo, E. Alegre, R. Alaiz-Rodríguez, F. Jáñez-Martino, and G. Azzopardi,
‘Assessment and estimation of face detection performance based on deep learning for forensic